KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

리스트 컴프리헨션

바꾸고 고르는 for를
대괄호 한 줄로 쓰기

```
prices = [4500, 5000, 6500, 3000]

[p + 500 for p in prices]
# [5000, 5500, 7000, 3500]

[p for p in prices if p >= 5000]
# [5000, 6500]
```

오늘의 순서

1 왜 필요한가 4–6쪽

2 만들기와 고르기 7–12쪽

3 여러 데이터에 쓰기 13–16쪽

4 함수와 함께 쓰기 17–19쪽

5 오류와 함정 20–23쪽

6 실습 24–26쪽

3쪽은 노트북 준비, 27–28쪽은 막힐 때와 한 장 요약이에요.

들어가기

22에서 이어서 해요

✓	리스트, 튜플, for, append, join	13
✓	딕셔너리 items(), 집합	15, 16
✓	if, 한 줄 if else	17
✓	range()	18
✓	lambda, map, filter, key=	21, 22

기억이 안 나면 왼쪽에 적힌 챕터를 먼저 다시 봐요.

노트북 다시 열기

1. VS Code로 hello-uv 폴더 열기
2. 탐색기에서 `basics.ipynb` 열기
3. 오른쪽 위 커널이 .venv인지 보기
4. 셀에 적고 `Shift` + `Enter` 로 실행

1. 왜 필요한가

for는 길고, map은 lambda까지 읽어야 해요

[1]
```python
prices = [4500, 5000, 6500]
up = []
for p in prices:
    up.append(p + 500)
print(up)
print(list(map(lambda p: p + 500, prices)))
```

← 세 줄을 매번 적어요

← 괄호를 풀어 가며 읽어요

```
[5000, 5500, 7000]
[5000, 5500, 7000]
```

```python
[p + 500 for p in prices]
```

← 한 줄인데 for처럼 읽혀요

1. 왜 필요한가

for의 세 조각을 대괄호 안으로 옮겨 적어요

for로 쓰면

```
up = []
for p in prices:
    if p >= 5000:
        up.append(p + 500)
```

컴프리헨션으로 쓰면

```
up = [p + 500          ← ① 넣을 값
      for p in prices  ← ② 꺼내기
      if p >= 5000]    ← ③ 고르기
```

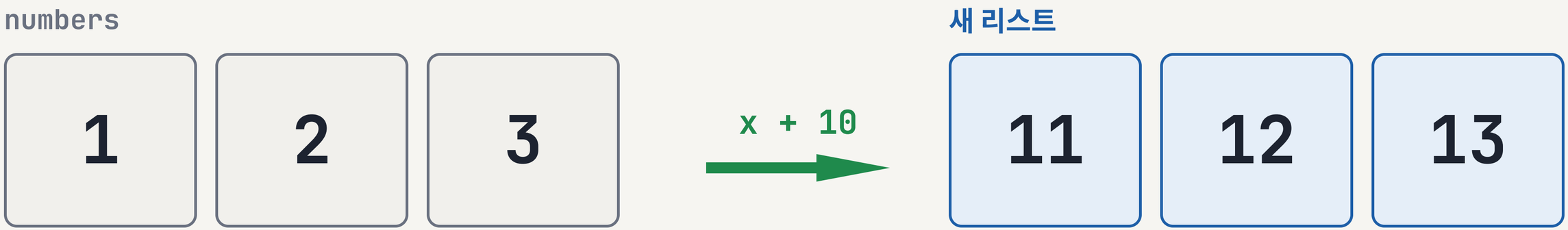
빈 리스트와 append는 대괄호가 맡고, for와 if의 콜론은 빼요.

1. 왜 필요한가

가장 짧은 컴프리헨션: 값마다 10 더하기

[2]
```python
numbers = [1, 2, 3]
print([x + 10 for x in numbers])
```

```
[11, 12, 13]
```
← 21의 list(map(lambda x: x + 10, numbers))와 같아요



[넣을 값 for 이름 in 리스트]는 값마다 계산한 새 리스트예요.

같은 일을 세 가지로 써요

[1]
```python
prices = [4500, 5000, 6500, 3000]
up = []
for p in prices:
    up.append(p + 500)
print(up)
print(list(map(lambda p: p + 500, prices)))
print([p + 500 for p in prices])
```

← 13의 for, 네 줄

← 21의 map, lambda까지

← 컴프리헨션, 한 줄

```
[5000, 5500, 7000, 3500]
[5000, 5500, 7000, 3500]
[5000, 5500, 7000, 3500]
```

← 결과는 셋 다 같아요

짧고 바로 읽히는 컴프리헨션을 가장 자주 써요.

2. 만들기와 고르기

뒤의 for부터 읽고, 앞의 값을 모아요

```
[p + 500 for p in prices]
```

② 계산해서 모아요　　① p를 하나 꺼내요

꺼낸 p	p + 500	모인 리스트
4500	5000	[5000]
5000	5500	[5000, 5500]
6500	7000	[5000, 5500, 7000]
3000	3500	[5000, 5500, 7000, 3500]

뒤에 if를 붙이면 골라 담아요

[2]
```python
big = []
for p in prices:
    if p >= 5000:
        big.append(p)
print(big)
print(list(filter(lambda p: p >= 5000, prices)))
print([p for p in prices if p >= 5000])
```

← 17의 골라 담기

← 21의 filter

← if를 뒤에 붙여요

```
[5000, 6500]
[5000, 6500]
[5000, 6500]
```

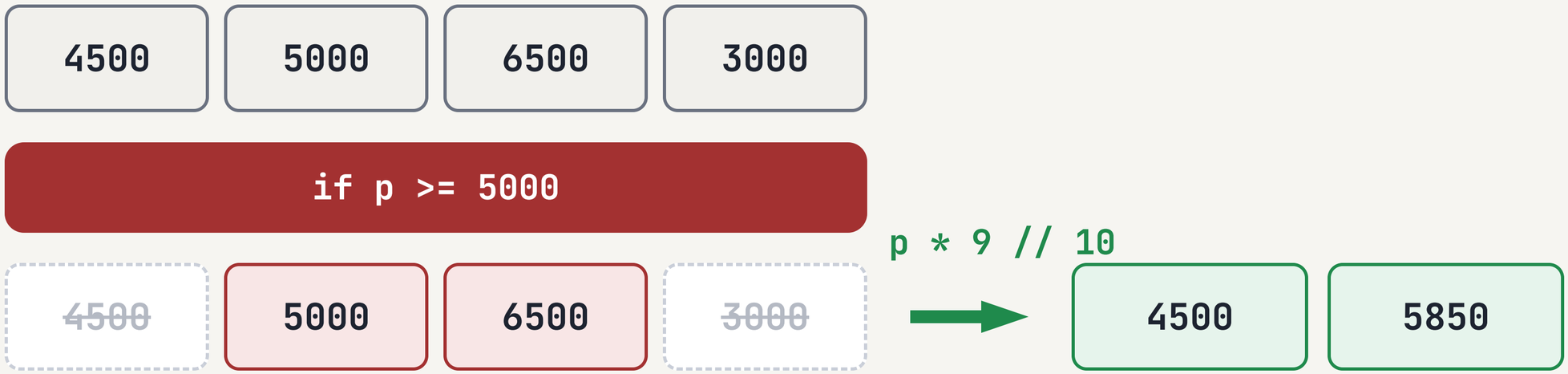
```python
[p for p in prices if p >= 5000]
```

← p는 그대로, 조건이 True인 p만 모여요

고르고 바꾸기를 한 번에 해요

[3]
```python
print([p * 9 // 10 for p in prices if p >= 5000])
print(list(map(lambda p: p * 9 // 10, filter(lambda p: p >= 5000, prices))))
```
컴프리헨션
21의 겹쳐 쓰기
Python

```
[4500, 5850]
[4500, 5850]
```



if로 남은 값만 앞의 식으로 바꿔서 모아요.

2. 만들기와 고르기

앞에 한 줄 if else를 넣으면 값을 바꿔 적어요

[4]
```python
print(["비쌈" if p >= 5000 else "보통" for p in prices])
```

```
['보통', '비쌈', '비쌈', '보통']
```

```python
["비쌈" if p >= 5000 else "보통" for p in prices]
```

↑ 17의 한 줄 if else는 값 하나라 넣을 값 자리에 들어가요

p	p >= 5000	넣을 값
4500	False	"보통"
5000	True	"비쌈"
6500	True	"비쌈"
3000	False	"보통"

2. 만들기와 고르기

if가 뒤에 있으면 고르기, 앞에 있으면 바꾸기

[5]
```python
print([p for p in prices if p >= 5000])          ← for 뒤
print([p if p >= 5000 else 0 for p in prices])   ← for 앞
```

```
[5000, 6500]
[0, 5000, 6500, 0]
```

자리	하는 일	else	개수
for 뒤의 if	고르기	안 써요	4개 → 2개
for 앞의 if else	바꾸기	꼭 써요	4개 → 4개

for보다 뒤인지 앞인지만 보면 돼요.

길이, 글자, range로도 만들어요

[1]
```python
names = ["latte", "tea", "coffee"]
print([len(m) for m in names])
print([m[0] for m in names])
print([n * n for n in range(1, 6)])
```

← 함수도 불러요. 21의 map(len, names)

← 13의 글자 꺼내기

← 18의 range로 숫자 만들기

```
[5, 3, 6]
['l', 't', 'c']
[1, 4, 9, 16, 25]
```

in 뒤에는 for로 돌 수 있는 것이면 다 와요.

튜플 리스트는 꺼내 담기로 이름을 붙여요

[2]
```python
orders = [("kim", "latte", 2), ("lee", "cake", 1), ("park", "coffee", 3), ("kim", "tea", 1)]
print([o[0] for o in orders])                                    번호로 꺼내기
print([menu_name for name, menu_name, qty in orders if qty >= 2])   이름 붙여 꺼내기
```

```
['kim', 'lee', 'park', 'kim']
['latte', 'coffee']
```

튜플 하나



13의 for name, menu_name, qty in과 같아요

qty >= 2처럼 조건도 이름으로 읽혀요.

중괄호와 키: 값으로 딕셔너리도 만들어요

[3]
```python
menu = {"coffee": 4500, "latte": 5000, "cake": 6500}
print({m: p * 11 // 10 for m, p in menu.items()})
stock = {"coffee": 3, "latte": 0, "cake": 2}
print({m: n for m, n in stock.items() if n > 0})
print([m for m in stock if stock[m] == 0])
```

← 값을 10% 올린 딕셔너리

← 재고 있는 것만 남기기

← 키만 필요하면 리스트

```
{'coffee': 4950, 'latte': 5500, 'cake': 7150}
{'coffee': 3, 'cake': 2}
['latte']
```

```python
{m: p * 11 // 10 for m, p in menu.items()}
```

← 키: 값을 앞에 적어요

{키: 값 for ...}는 딕셔너리, [값 for ...]는 리스트예요.

중괄호에 값만 적으면 집합이 돼요

[4]
```python
buyers = {o[0] for o in orders}
print(len(buyers))
print(sorted(buyers))
```

```
3
['kim', 'lee', 'park']
```

← sorted로 줄 세워 찍었어요

o[0]을 모으면



↑ kim은 이미 있어서 빠져요

집합을 찍는 순서는 실행할 때마다 달라질 수 있어요.

4. 함수와 함께 쓰기

sum, max에 바로 넘겨요

[1]
```python
menu = {"coffee": 4500, "latte": 5000, "cake": 6500, "tea": 4000}
cart = ["coffee", "cake", "latte", "latte"]
print(sum([menu[m] for m in cart]))   ← 장바구니 가격 합계 21000
print(max([len(m) for m in menu]))    ← 가장 긴 메뉴 이름의 길이 6
```
Python

```
21000
6
```

19의 calc_total에서는 세 줄

```python
total = 0
for m in cart:
    total += menu[m]
```



```python
sum([menu[m] for m in cart])
```

계산할 값만 리스트로 모아 sum, max, len에 넘겨요.

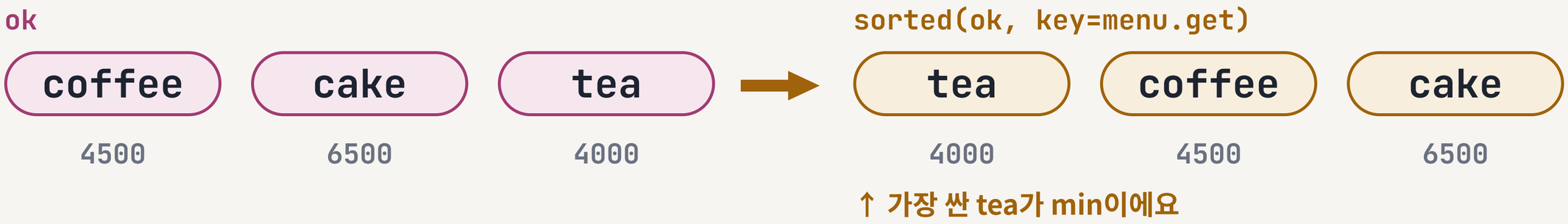
고른 다음 22의 sorted, min으로 이어요

[2]
```python
stock = {"coffee": 3, "latte": 0, "cake": 2, "tea": 5}
ok = [m for m in menu if stock[m] > 0]
print(ok)
print(sorted(ok, key=menu.get))
print(min(ok, key=menu.get))
```

← 품절 아닌 메뉴만 골라요

← 22의 key로 가격순

```
['coffee', 'cake', 'tea']
['tea', 'coffee', 'cake']
tea
```



4. 함수와 함께 쓰기

이럴 땐 for가 더 읽기 쉬워요

이럴 때	쓰는 것
새 리스트 하나를 만들 때	**컴프리헨션**
print로 찍기만 할 때	for
한 번에 여러 줄을 할 때	for
break로 멈춰야 할 때	for
조건이 두 개를 넘을 때	for나 def

한 줄이 길어지면 for로 풀어 써요.

참고, 읽기만

[3]
```python
scores = [[90, 80], [70, 60]]
print([x for row in scores
       for x in row])
```

```
[90, 80, 70, 60]
```

for를 두 번 적었어요. 앞의 for가 바깥 반복이에요.

5. 오류와 함정

뒤의 if에 else를 붙이면 SyntaxError가 나요

[1]
```python
prices = [4500, 5000, 6500, 3000]
print([p for p in prices if p >= 5000 else 0])
```
뒤의 if에 else

Python

```
SyntaxError: invalid syntax
```

읽는 법: for 뒤의 if는 고르기라서 else를 받지 않아요.

고르기만 하려면 else를 빼요

× ~~`[p for p in prices if p >= 5000 else 0]`~~ → ✓ `[p for p in prices if p >= 5000]`

모두 바꾸려면 if else를 for 앞으로 옮겨요

× ~~`[p for p in prices if p >= 5000 else 0]`~~ → ✓ `[p if p >= 5000 else 0 for p in prices]`

SyntaxError는 실행 전에 나서 첫 줄 prices도 만들어지지 않아요.

5. 오류와 함정

print를 넣으면 None 리스트가 생겨요

[2]
```
[print(m) for m in ["latte", "tea"]]
```
Python

```
latte
tea
```
← 찍히기는 해요

```
[None, None]
```
← print가 돌려준 None이 모였어요

읽는 법: 컴프리헨션은 값을 모으는 곳이에요. 찍기만 할 때는 for를 써요.

× ~~`[print(m) for m in ["latte", "tea"]]`~~ → ✓

```
for m in ["latte", "tea"]:
    print(m)
```

셀 마지막 줄이라 [None, None]이 결과로 보였어요.

5. 오류와 함정

소괄호로 감싸면 리스트가 아니에요

[3]
```
prices = [4500, 5000, 6500, 3000]
print((p + 500 for p in prices))
print(list(p + 500 for p in prices))
```
Python

```
<generator object <genexpr> at 0x7f2ba871b440>
```
← 튜플이 아니에요

```
[5000, 5500, 7000, 3500]
```
← list로 감싸야 보여요

읽는 법: 소괄호는 달라고 하면 값을 하나씩 꺼내 주는 묶음이에요. 21의 map object와 비슷해요.

× ~~(p + 500 for p in prices)~~ → ✓ `[p + 500 for p in prices]`

0x 뒤 숫자는 메모리 위치라서 화면마다 달라요.

5. 오류와 함정

앞과 뒤의 이름이 다르면 NameError가 나요

[4]
```
print([x + 10 for p in prices])
```
Python

```
NameError: name 'x' is not defined
```
← x라는 변수는 없어요

```
[x + 10 for p in prices]
```

↑ 앞의 값은 x

↑ 꺼낸 이름은 p

× ~~`[x + 10 for p in prices]`~~ → ✓ `[p + 10 for p in prices]`

앞 예제를 복사해서 뒤쪽만 고칠 때 자주 나요.

6. 실습

장바구니 영수증을 컴프리헨션으로 찍어요

[1]
```python
menu = {"coffee": 4500, "latte": 5000, "cake": 6500}
cart = ["latte", "cake", "coffee"]
lines = [f"{m:<7}{menu[m]:>6,}원" for m in cart]
print("\n".join(lines))
print(f"합계   {sum([menu[m] for m in cart]):>6,}원")
```

← 메뉴마다 한 줄씩

← 합계는 가격 리스트를 sum

```
latte    5,000원
cake     6,500원
coffee   4,500원
합계    16,000원
```

← 13의 칸 맞추기 f-string도 넣을 값이에요

21에서 def와 map으로 한 일을 한 줄로 했어요.

6. 실습

품절 목록과 주문 가능 메뉴판을 만들어요

[2]
```python
menu = {"coffee": 4500, "latte": 5000, "cake": 6500, "tea": 4000}
stock = {"coffee": 3, "latte": 0, "cake": 2, "tea": 0}
soldout = [m for m in stock if stock[m] == 0]
print("품절:", ", ".join(soldout))
board = {m: menu[m] * 11 // 10 for m in menu if stock[m] > 0}
print(board)
```

← 품절 리스트

← 새 메뉴판 딕셔너리

```
품절: latte, tea
{'coffee': 4950, 'cake': 7150}
```



6. 실습

성적표에서 합격자, 결과표, 평균, 1등을 뽑아요

[3]

```python
scores = [["kim", 92], ["lee", 78], ["park", 85], ["choi", 59]]
print([name for name, score in scores if score >= 80])
print(["합격" if score >= 60 else "재시험" for name, score in scores])
print(sum([score for name, score in scores]) / len(scores))
print(max(scores, key=lambda s: s[1])[0])
```

Python

← 뒤의 if: 80점 이상 고르기
← 앞의 if else: 모두에게
← 점수만 모아 평균
← 22의 key로 1등

```
['kim', 'park']
['합격', '합격', '합격', '재시험']
78.5
kim
```

고르기, 바꾸기, 함수와 함께 쓰기를 한 셀에서 다 썼어요.

정리

막히면 오류의 마지막 줄을 봐요

`SyntaxError: invalid syntax` 뒤의 if에 else를 붙였어요	✓ else는 for 앞으로
`NameError: name 'x' is not defined` 앞과 뒤의 이름이 달라요	✓ 같은 이름으로
[None, None]이 같이 나와요 대괄호 안에서 print를 했어요	✓ 찍기는 for로
<generator object ...>가 나와요 소괄호로 감쌌어요	✓ 대괄호로
개수가 줄었어요 for 뒤에 if를 붙여 골랐어요	✓ 바꾸려면 if else를 앞에
한 줄이 너무 길어요 조건이나 할 일이 많아요	✓ for나 def로 나누기
집합 순서가 슬라이드와 달라요 집합은 순서가 없어요	✓ sorted()로 찍기

헷갈리면 같은 일을 for 문으로 먼저 써 보고 세 조각을 옮겨요.

한 장 요약

`[x + 10 for x in xs]`

값마다 바꾼 리스트

`{k: v for k, v in d.items()}`

딕셔너리 만들기

`[x for x in xs if x > 0]`

조건에 맞는 값만 고르기

`{x for x in xs}`

집합 만들기

`[a if c else b for x in xs]`

앞의 if else는 바꾸기

`sum([...]), max([...])`

모은 리스트를 바로 넘기기

`[n for n, s in pairs]`

튜플은 꺼내 담기로

for 문

찍기, 여러 줄, break는 for로

뒤의 if는 고르기, 앞의 if else는 바꾸기예요.

다음: 24에서 클래스로 붕어빵 틀을 직접 만들어요.